

Two hours**THE UNIVERSITY OF MANCHESTER****INTRODUCTION TO PROGRAMMING 2 (ONLINE)****Sample Exam**

Instructions for candidates

Answer ALL questions and upload your solutions to Gradescope. The examination is divided into TWO sections and is worth a total of 40 marks:

- Section 1 is worth 25 marks (62.5%)
- Section 2 is worth 15 marks (37.5%)

While you will have had access to Section 1 in advance, you must implement all code from scratch during the exam.

You are permitted to make notes on the question paper, but you must not remove any materials from the exam room.

Any attempt to access material from outside exam conditions is not permitted.

Additional permitted materials

Answer this exam on the computer provided. You should use the installed Java compiler to test your code before uploading. The use of calculators is NOT permitted.

Access to the Java API (and JavaFX APIs where appropriate) will be possible in the exam as linked from Canvas. Other websites will NOT be available during this exam.

You have been provided with a supplement called “*Working on the Windows Assessment Desktop*”, this outlines the allowed applications, **Notepad++**, **Firefox**, **Powershell**, **javac**, **jar** and how to use them for this exam.

Uploading your responses to Gradescope

You should write all of your code in a single package, the corresponding folder is to be uploaded as a zip file. For example, if the exam requires you to write a class named Book in the **library** package, and you've been working on the file S:\library\Book.java then from S:\ you should zip the **library** directory and upload that zip file to Gradescope.

You can upload your solutions as many times as you wish during the exam. Each time you upload a solution, your mark/test output will be updated.

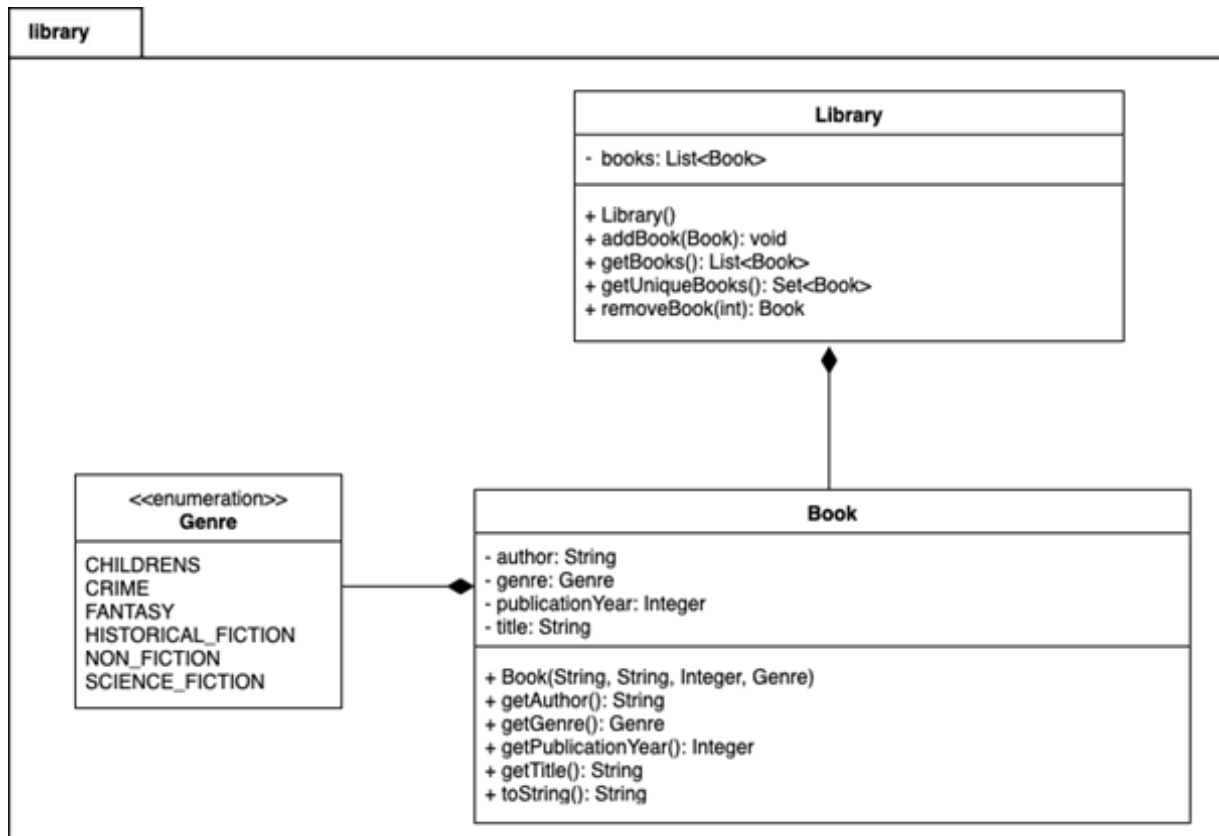
Your final mark will be determined by the last solution that you upload to Gradescope, but note the mark calculation may occur after the end of the exam. You **MUST** upload a solution to Gradescope before the exam ends in order to receive a non-zero mark.

© 2026 The University of Manchester

Section 1. Answer ALL PARTS of BOTH questions in this section.

The following UML diagram represents entities involved in a library management system.

Note: Where an ancestor type is specified in the UML (in this case, List, and Set), this exact type must be used (a different type may be used for instantiation).



Question 1

- (a) Implement the **Genre** entity in the above UML diagram, respecting its specification as an enum type.

[1 mark]

- (b) For the **Book** class, implement:

- i. The class signature (i.e., the visibility modifier, class name, opening and closing braces).
- ii. All four of the specified attributes. You should assume that author Strings will be given in the form "<firstname> <lastname>", e.g. "Sarah Clinch".
- iii. The `getAuthor` method, a getter for the author attribute.
- iv. The `getGenre` method, a getter for the genre attribute.
- v. The `getPublicationYear` method, a getter for the publicationYear attribute.
- vi. The `getTitle` method, a getter for the title attribute.
- vii. The `toString` method, which returns in the format "<title> (<author>)", e.g. "Java: Just in Time (John Latham)".
- viii. The constructor, which takes parameters corresponding to its attributes in the following order: title, author, publicationYear, genre.

[5 marks]

(c) For the **Library** class, implement:

- i. The class signature (i.e., the visibility modifier, class name, opening and closing braces).
- ii. The specified attribute
- iii. The `getBooks` method, a getter for the `books` attribute.
- iv. The `addBook` method, which appends the specified book to the `books` attribute.
- v. The `removeBook` method, which removes and returns the book at the specified index of the `books` attribute. If presented with an index that is out of range, a `java.lang.IndexOutOfBoundsException` should be thrown.
- vi. The constructor, which *instantiates* (but does not populate) the `books` attribute as an `ArrayList`.

Note that the library may own more than one copy of popular books. For this reason, the library's inventory of books may contain multiple **Book** objects with the same author and title.

[4 marks]

Question 2

As previously stated, the library may own more than one copy of popular books.

You can assume that two books published with the same author and title are equivalent. This comparison should be case sensitive and include spaces – i.e. “Everyday words” should not be considered equal to “Every day words”, nor should “Sarah Clinch” be considered equal to “Sarah CLINCH”.

- (a) Add an `equals` method to the **Book** class (overriding the existing method of the same name in the `Object` class). This method should return `true` if the author and title of the specified book is the same as the author and title of the current book.

[2 marks]

- (b) The hashCode method is used to generate a hashed value (an int) representing an object for use in certain data structures (e.g. `java.util.HashSet`).

Objects that are considered equal by the equals method should return the same hash code (i.e. the same int value). Different objects do not need to return different hash codes, although they often do.

Add a hashCode method to the **Book** class (overriding the existing method of the same name in the Object class). This method should return the same value for Books that have the same title and author, but must NOT always return the same value for books that do not (i.e. you cannot return a constant value).

[2 marks]

- (c) The Set data structure should be used to implement the getUniqueBooks method. The return value should contain exactly one entry for each book that the **Library** has at least one copy of (i.e., the value of getBooks, but with all duplicate entries removed). Implement the getUniqueBooks method of the **Library** class as specified in the UML diagram. The method should NOT affect the contents of the books attribute.

[3 marks]

Question 3

Add a new method `findFictionBooks` to the **Library** class. The method should have the following signature:

```
public Stream<Book> findFictionBooks (  
    Integer publishedInOrAfter, String searchTerm  
)
```

This method should use **STREAMS** to return all books that:

- Do **NOT** have a genre of `NON_FICTION`, **AND**
- Have a `publicationYear` that is equal to or greater than the year specified in the `publishedInOrAfter` parameter, **AND**
- Have a title that contains the search term specified in the `searchTerm` parameter (case insensitive)

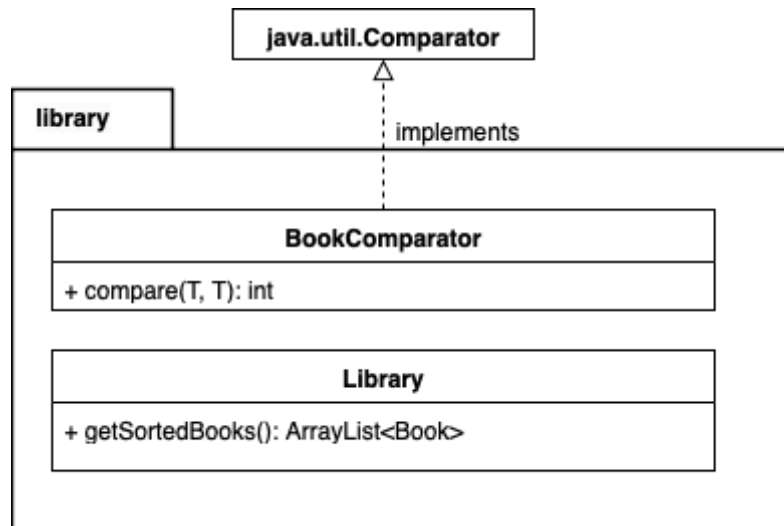
If a `null` parameter is provided for `publishedInOrAfter` then all books matching the `searchTerm` should be returned. Likewise, if a `null` or `empty` `searchTerm` is provided, all books matching the `publishedInOrAfter` filter should be returned.

[8 marks]

Section 2. Answer BOTH questions in this section.

Question 1

The following UML shows some additions to the *library* package.



As a reminder, a compare method returns a negative integer (typically -1) if the first parameter is “less than” the second parameter, zero if the two parameters are equal, and a positive integer (typically +1) if the first parameter is “more than” the second parameter

Note: Unlike the equals and hashCode methods set out in **Section 1 Question 2**, the **BookComparator** will NOT treat all books by the same author with the same title as being equal.

(a) Implement the **BookComparator** to compare books by:

1. title (case-insensitive).
2. Then (if the titles are equal) by author (case-insensitive based on "<lastname>, <firstname>")

Note: the REORDERING of the two parts of the author name.

3. Then (if both previous fields are equal) by year. You should assume that more recent books take precedence over older ones (i.e., "2025".compareTo("1985"); will return **-1**).
4. Finally, (if all three previous fields are equal) by genre name (alphabetic, using the names indicated in the **Section 1 Question 1** UML diagram)

In the case of title, author and genre, comparisons of alphabetic AND non-alphabetic String characters should be consistent with the behaviour of `String.compareTo` (i.e., both `"a".compareTo("b")`; and `"ab".compareTo("a")`; will return a negative value).

[10 marks]

(b) Implement the `getSortedBooks` method so that it returns books sorted in accordance with the constraints set out in part (a).

Note that

- This method should NOT return duplicate entries (i.e. you should use both the **BookComparator** and the previously implemented `getUniqueBooks` method).
- The method should NOT alter the ordering of the **Library**'s books attribute.
- Some conversion between data structures will be needed to ensure a return value of the specified type.

[5 marks]

END OF EXAMINATION